Lappeenrannan teknillinen yliopisto

School of Engineering Science

Sofware Development Skills

Md Mahi Al Jubair Talukder, 001166325

LEARNING DIARY, INTRODUCTION MODULE

10.5.2026
I began watching the Android development tutorial to get a solid grasp of the mobile development fundamentals. The main focus of the first section was setting up the development environment properly. I installed Android Studio and configured the necessary SDKs for the project. I also set up an Android Virtual Device (emulator) so I could test the application locally. I learned about the basic project structure, specifically the differences between the resource directories and the main Java source files. I successfully ran the initial "Hello World" build to ensure my environment was working correctly before moving on.

10.5.2026
I continued with the tutorial, diving into user interface creation. The video explained how to build layouts, and I learned about organizing basic components like text views, buttons, and columns. I tried to arrange the UI elements exactly as shown in the video, but I ran into some layout alignment issues where my components were overlapping on the screen. I paused the tutorial, searched through the official Android developer documentation, and realized I needed to apply the correct constraint attributes for padding and spacing. I was able to fix the layout issue and pushed my first major commit to my Git repository.

10.5.2026
In the final part of my learning session, the focus shifted to handling user interaction and logic implementation. It was really interesting to learn how the UI is designed to react to data changes using Java. I implemented a simple interactive feature in the app following the instructor's guidance. I encountered a small bug where the UI wasn't updating when a button was clicked because I had incorrectly referenced the resource ID. Rewatching that specific segment of the video clarified the correct syntax for me. This hands-on practice helped solidify my understanding of how data flows within an Android application.

13.5.2026
Today, I moved on to building a practical "Addition App" based on the Java tutorial. This was a great exercise in connecting the visual UI with backend logic. I used a ConstraintLayout to anchor two Plain Text (EditText) inputs, an "Add" button, and a large TextView for the result. I learned the importance of naming IDs descriptively (like `addBTN`) to keep the Java code organized.

In the `MainActivity.java` file, I practiced using `findViewById` to link my UI components to Java objects. The most important part was implementing the `setOnClickListener` for the button. I had to remember to parse the input text into integers using `Integer.parseInt()` before performing the calculation, and then concatenate the result with an empty string to convert it back for the `setText()` method. The tutorial also introduced me to the Debugger; I learned how to set breakpoints and use 'Step Over' (F8) to watch variables in the console as the app runs. This systematic approach to finding errors is definitely a skill I'll need as my projects get more complex.

LEARNING DIARY, CORE ELEMENTS MODULE

10.5.2026

I started the second lecture, which focuses on building a "Quick App Launcher" to learn how to navigate between multiple screens. The core concepts introduced today were **Activities** and **Intents**. I learned that an Activity is essentially a single screen or rectangular area that displays something, while an Intent acts as a messenger or translator requesting an action to be performed, like reacting to a user's tap. I created the new project and set up my Android Studio preferences, specifically turning on **"Auto Import"** and line numbers to make coding easier and prevent missing class errors. I also learned best practices for using the `strings.xml` resource file to manage the app's display text, which is better for organization and future translations than hardcoding strings directly into the layout.

11.5.2026

Today I focused on the visual design and creating new screens. In the visual designer, I built the main layout by dragging in two buttons: one meant to launch a "Second Activity" and another for a "Google" search. I practiced using the **Constraint Layout** to align the buttons relative to each other and the screen, which ensures the layout maintains a consistent look whether the device is held in portrait or landscape mode. After finishing the main menu, I learned how to add a completely new screen by creating a second Empty Activity in the Java project folder, and I added a simple, centered TextView to its layout file.

11.5.2026

I moved into the Java code today to make the "Second Activity" button functional. Just like in the first app, I used `findViewById()` and an `OnClickListener` to link the UI button to my code. The most exciting part was using an Intent to not only switch screens but also pass data between them. I created an Intent object targeting the `SecondActivity.class` and used the `putExtra()` method to bundle a key-value pair containing the text "HELLO WORLD!". In the Second Activity's Java file, I wrote logic to retrieve the Intent, check if it had extra information, extract the string using my custom key, and update the TextView to display it. Seeing the text update successfully on the new screen was highly rewarding!.

11.5.2026

In my final session for this module, I learned how to use Intents to launch completely external applications. For the "Google" button, I created an **Action View Intent** designed to go outside of my app. I set up a string with a web address ("www.google.com") and parsed it into a URI object. The instructor taught a crucial safety check: before attempting to start the external activity, I used an `if` statement with `resolveActivity()` and the `getPackageManager()` to verify that the Android device actually has a browser capable of handling the web link. I ran the app in the emulator, and it successfully opened the default browser to Google. This lecture really solidified my understanding of how Intents handle communication both internally between app screens and externally with other apps.

LEARNING DIARY, LISTS LAYOUTS AND IMAGES MODULE

12.5.2026
Today I started the third lecture, which focuses on building a "List App" to learn about lists and displaying images. My first task was to add a **ListView** container to the main XML layout. Instead of hardcoding my data directly into the Java file, I learned a much cleaner way to organize information using the `strings.xml` resource file. I created three separate **string arrays** to hold data for a peach, a tomato, and a squash, creating parallel lists for the item names, their prices, and their descriptions.

13.5.2026
I focused on customizing the appearance of my list today. I didn't want just a simple text list, so I created a custom layout file called `my_list_view_detail.xml` using a **Relative Layout** to position three `TextView` components (name, description, and price) relative to each other. The most challenging part was connecting my string arrays to this custom layout. I had to build a custom Java class called **ItemAdapter** that extends `BaseAdapter`. Inside this adapter, I used a **LayoutInflater** to take my custom XML template and generate it for every single row in the list, automatically filling the text views with the correct string data.

14.5.2026
Today was all about making the list interactive. I attached an `setOnItemClickListener` to my ListView so the app can react when a user taps a specific row. Building on the concepts from the second lecture, I created an Intent to launch a second screen called `DetailActivity`. The coolest part was using the `putExtra()` method to pass the exact *index* of the tapped item (0, 1, or 2) over to the new screen, ensuring the second activity knows exactly which item's details to display.

15.5.2026
In my final session for this module, I focused on visual assets and memory management. I added photos of the produce into the `drawable` folder and set up an **ImageView** on the second screen. The instructor taught a critical lesson today: high-resolution images can consume too much memory and crash the app. To prevent this, I wrote a custom method using `BitmapFactory.Options` to trace the boundaries of the image without actually loading it into memory. By comparing the image width to the device's screen width, I calculated a scale ratio, and then used `inSampleSize` to decode and display a perfectly scaled-down version of the bitmap. This was a complex but incredibly valuable technique for building robust mobile apps!

LEARNING DIARY, FINAL PROJECT MODULE

17.5.2026

Today, I started the final project for the course, building the mobile interface for the Ravintola Restaurant ecosystem. Moving away from basic separate screens, I implemented a Fragment-based modular architecture using the modern Jetpack Navigation Component. I focused on setting up the main user interface with Material Design 3 (M3) guidelines, integrating a `BottomNavigationView` to allow users to switch seamlessly between the Menu, Cart, and Profile without launching entirely new activities. This approach felt much more professional compared to the explicit Intents I used in earlier modules.

18.5.2026

My focus today shifted to backend integration. The app needed to dynamically fetch menu data from our live RESTful API hosted on Render instead of relying on local hardcoded arrays. I implemented Retrofit 2 and OkHttp 3 to handle asynchronous network calls, alongside the GSON library to automatically convert JSON responses into typed Java objects. I created an `ApiClient` singleton to manage the base URL connection. Learning how to execute API calls asynchronously on a background thread to prevent the main UI from freezing was a major breakthrough.

19.5.2026

Today was dedicated to user management and secure authentication. I built out the Login and Register fragments to communicate with our Node.js `/api/users` endpoints. The most challenging aspect was securely handling user sessions. I created a custom `TokenManager` class to extract and locally store the JSON Web Tokens (JWT) returned by the server. This guarantees that a user remains logged in across sessions and that their token is automatically attached as an authorization header when accessing protected information like order history.

20.5.2026

I spent the day developing the app's standout feature: the interactive Custom Pizza Builder. I utilized `RecyclerView` with custom item adapters to display available ingredients efficiently, a significant step up from the basic `ListView` covered in the third lecture. The state logic here was complex; the UI had to dynamically recalculate the total price in real-time as users selected their pizza size (8", 10", 12") and toggled different sauces, meats, and veggies. Additionally, to handle high-resolution menu images without memory crashes, I integrated the Glide library to automatically manage image caching and rendering over the network.

21.5.2026

In my final session, I focused on the shopping cart system and the order submission flow. I engineered a persistent local cart that allows users to review items and adjust quantities

before checkout. Constructing the correct JSON payload for the POST request to `/api/orders` required careful planning, as the array needed to support both standard menu IDs and complex custom pizzas with nested ingredient arrays. I successfully tested the checkout for both authenticated accounts and guest users. Seeing the order reach the cloud PostgreSQL database and trigger a receipt snapshot brought the entire project together perfectly.